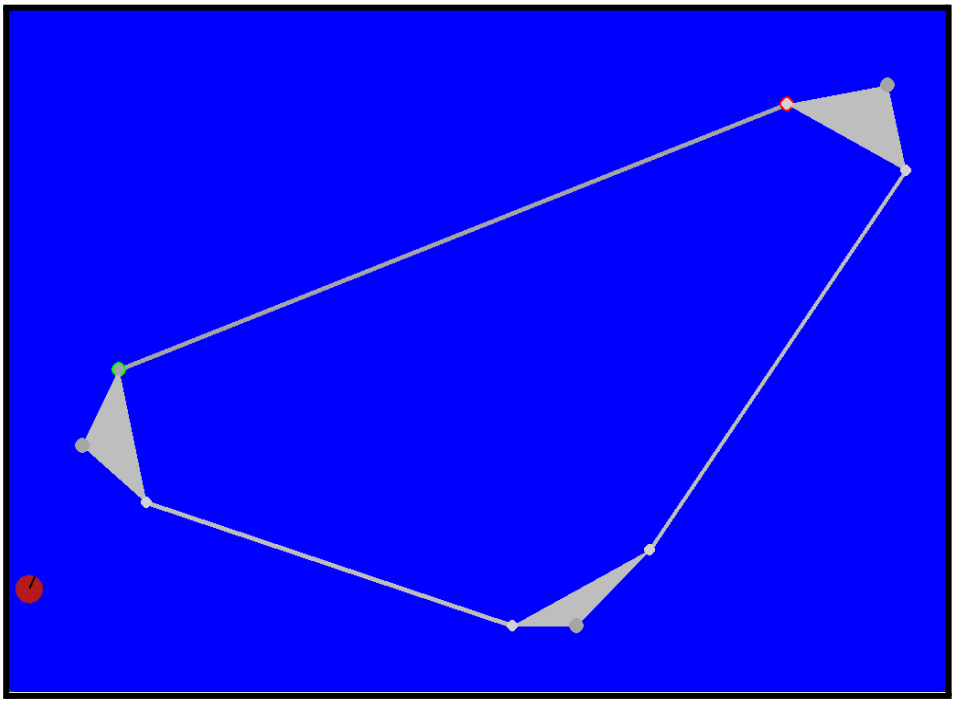
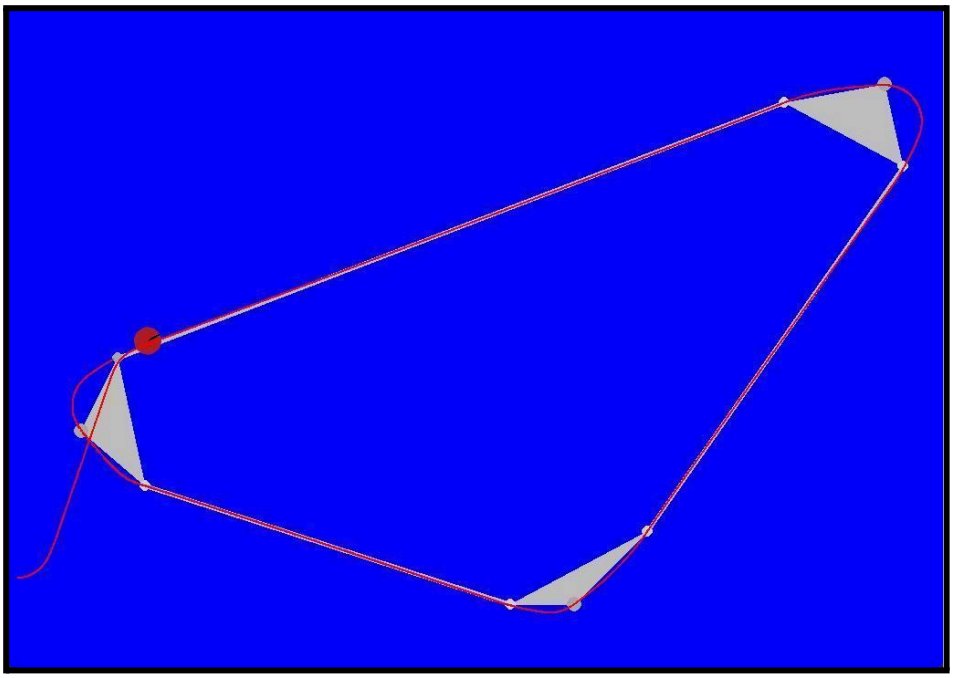
Razonamiento y Representación del Conocimiento: Práctica 1





Índice:

1. Introducción y contexto....3
1.1. Descripción de la práctica:....3
1.2. Datos y comportamiento del robot:....3
1.3. Tipos de objetivos y puntuación:....3
2. Ejercicio 1: Sistema experto....4
2.1. Evolución del trabajo:....4
2.1.1. Primera versión y avances iniciales:....4
2.1.2. Proceso de optimización:....4
2.1.3. Historial de puntuaciones:....5
2.1.4. Ideas descartadas:....5
2.1.5. Pruebas con otros circuitos y último cambio:....5
2.2. Explicación del código final:....6
2.2.1. Estructura y variables de la clase:....6
2.2.2. Funciones auxiliares y geométricas:....6
2.2.3. Función iraobjetivo():....8
2.2.4. Función tomarDecision():....9
2.3. Resultados del modelo final:....10
2.4. Conclusiones y posibles mejoras:....12
2.4.1. Ventajas y puntos fuertes:....12
2.4.2. Limitaciones y futuras mejoras:....12
3. Ejercicio 2: Sistema experto difuso....13
3.1. Evolución del trabajo:....13
3.1.1. Primera versión y avances iniciales:....13
3.1.2. Proceso de optimización:....13
3.1.3. Historial de puntuaciones:....15
3.1.4. Ideas descartadas:....15
3.2. Explicación del código final:....15
3.2.1. Estructura y variables de la clase:....15
3.2.2. Funciones auxiliares:....15
3.2.3. Variables y conjuntos difusos:....16
3.2.4. Reglas difusas:....17
3.2.5. Motor de inferencia:....18
3.2.6. Función tomarDecision():....19
3.2.7. Cosas que tener en cuenta con la librería:....20
3.3. Resultados del modelo final:....20
3.4. Conclusiones y posibles mejoras:....21
3.4.1. Ventajas y puntos fuertes:....21
3.4.2. Limitaciones y mejoras futuras:....21
4. Comparación entre los dos ejercicios y conclusiones generales....22
5. Datos de puntuaciones....23
5.1 ExpertSystem (ejercicio 1):....23
5.2 FuzzySystem (ejercicio 2):....23
6. Uso de la IA....24
7. Bibliografía....24
8. Repositorio de GitHub....24

1. Introducción y contexto

1.1. Descripción de la práctica:

El objetivo de esta práctica es controlar un robot móvil que se mueve por un plano sin obstáculos, dándole una velocidad lineal y una velocidad angular. Lo único que sabemos de él es su pose: la posición (x, y) y la orientación.

Con esos datos, el sistema tiene que llevar al robot hasta el final de cada objetivo, intentando que vaya lo más pegado posible al camino y que tarde lo menos posible. Cuanto mejor lo haga, más puntuación de recompensa le da el simulador.

La práctica tiene dos ejercicios, que resuelven el mismo problema de dos formas distintas:

- **Ejercicio 1: sistema experto clásico.** Reglas if-then, en el fichero expertSystem.py (clase ExpertSystem).
- **Ejercicio 2: sistema experto difuso.** Reglas difusas con la librería fuzzy-expert, en el fichero fuzzyExpert.py. Parte de la lógica del ejercicio 1.

1.2. Datos y comportamiento del robot:

El robot es de tracción diferencial, con dos ruedas motrices. Esto quiere decir que cuando la velocidad lineal es 0, el robot gira sobre su propio centro. Sus límites son:

- **Velocidades máximas:** lineal de 3 m/s y angular de 1 rad/s.
- **Aceleración y deceleración:** lineal de 1 m/s^2 y angular de 0.5 rad/s^2.

Esto último es importante: el robot no cambia de velocidad al momento, lo hace poco a poco (tiene inercia). Si le pedimos que frene o que gire, tarda un rato en hacerlo, y por eso si va rápido se pasa de los puntos.

En cada momento el programa recibe la pose del robot y tiene que devolver la velocidad lineal y la angular que queremos darle. La pose llega con 5 valores (x, y, orientación, velocidad lineal actual y velocidad angular actual), pero yo solo uso los tres primeros. La orientación está en grados.

1.3. Tipos de objetivos y puntuación:

El simulador puntúa dos cosas: lo cerca que va el robot del camino y lo rápido que llega. Para entender cómo funciona la puntuación (sobre todo lo del tiempo), le he preguntado a la IA:

- **Tramo en recta (segmento):** tiene un punto de inicio y uno de fin. En cada instante, si el robot está a menos de 3 metros de la recta que une los dos puntos, suma puntos, y más cuanto más cerca esté (si está a menos de 1 cm suma 100). Al final, esa suma se divide entre $(1 + \text{tiempo})^3$, así que el tiempo pesa muchísimo.
- **Tramo de triángulo:** tiene tres puntos (inicio, punto medio y fin). Hay que ir del inicio al final rodeando el punto medio, sin entrar en el triángulo. La puntuación empieza en 500 y resta un punto por cada instante que el robot está dentro del triángulo. Además, solo cuenta si el robot pasa más cerca del punto medio que la altura del triángulo (si no, la nota sale negativa). Al final se divide entre $(1 + \text{tiempo})^2$.

- **Cambio entre objetivos:** el simulador no pasa al siguiente tramo hasta que el robot llega a menos de 0.5 metros del punto final del objetivo actual. En ese momento calcula la nota del tramo y empieza a contar el tiempo del siguiente.

2. Ejercicio 1: Sistema experto

En este ejercicio hago un sistema experto con reglas “if-then” (fichero expertSystem.py). Con la versión final tengo **33.81 puntos** en el recorrido 1. Primero cuento cómo ha ido el trabajo, después explico el código, y por último pongo los resultados y las conclusiones.

2.1. Evolución del trabajo:

En este apartado me voy a centrar en cómo ha ido mejorando el sistema experto, desde la primera versión hasta la final.

2.1.1. Primera versión y avances iniciales:

Al empezar, lo primero que hice fue mirar el código que nos dieron en Moodle y entender qué significaba cada variable del simulador. Después de revisar los archivos, vi que lo básico que necesitaba era esto:

- **Pose del robot:** llega en cada instante en poseRobot. poseRobot[0] es la coordenada X, poseRobot[1] la coordenada Y y poseRobot[2] la orientación.
- **Objetivos:** con el método setObjetivo guardo el objetivo activo en self.objetivoActual, y desde ahí puedo sacar sus puntos de inicio y fin.

Para probar cómo se movía el robot, imprimí por la terminal la posición y el ángulo mientras corría el simulador. Con eso me dí cuenta de que lo que tenía que hacer era calcular hacia qué ángulo está el objetivo y girar el robot poco a poco hasta que mire hacia él.

Para sacar ese ángulo uso atan2 con las diferencias de posición (ΔY y ΔX) y paso el resultado de radianes a grados. También me di cuenta de que cuando el robot daba muchas vueltas su orientación seguía creciendo (360, 720...), y así era un lío decidir hacia qué lado girar. Por eso resto el ángulo del objetivo menos la orientación del robot y dejo el resultado entre -180° y +180°. Así sé siempre cuál es el giro más corto.

Con condiciones sencillas de velocidad conseguí que el robot diera una vuelta entera al circuito. Iba bastante mal (se pasaba de los puntos y todavía no tenía en cuenta los triángulos), pero me sirvió para empezar. Con esta primera versión saqué **9.63 puntos**.

2.1.2. Proceso de optimización:

Con la primera versión funcionando, empecé a arreglar los fallos y a mejorar la puntuación. Estos fueron los cambios:

- **Detectar el tipo de objetivo:** en vez de decirle yo al código qué tipo de tramo es cada uno, uso getType() sobre self.objetivoActual. Así el código sabe solo si el tramo es un segmento (tipo 1) o un triángulo (tipo 2), y vale para cualquier circuito.
- **Gestionar los triángulos:** como tienen 3 puntos (inicio, fin y un punto medio para facilitar el giro), utilizo el punto medio para que el robot rodee el triángulo y no pierda puntuación. Primero va hacia el punto medio y, cuando llega cerca, cambia el destino al punto final.

- **Limpiar el código:** quité las funciones repetidas que usaba al principio (distanciaInicial y distanciaFinal) y dejé una sola función para las distancias y otra para los ángulos, que se usan siempre.
- **Cambiar de punto antes de llegar:** si esperaba a estar justo encima del punto para cambiar de destino, el robot hacía giros muy bruscos y se pasaba por la inercia. Ahora cambio de destino un poco antes: en el primer objetivo cuando la distancia es de 3.5 o menos, y en el resto cuando es de 2 o menos (esta distancia es la suma |ΔX| + |ΔY|, no la real). Así el robot empieza a girar antes y la curva sale más suave.
- **La desviación del primer tramo:** al meterse en la primera recta larga, el robot se desviaba hacia abajo y hacía una curva muy grande. Pasaba porque iba a 3 m/s sin estar bien alineado, y con esa velocidad el radio de giro es enorme. Lo arreglé con dos fases: al cruzar el inicio mantengo una velocidad baja (1 m/s) hasta que el error de ángulo es de 10° o menos, y solo entonces dejo los 3 m/s. Con esto el robot ya no se desvía y va por el centro de la recta.

2.1.3. Historial de puntuaciones:

En esta tabla pongo cómo fue subiendo la nota. Al principio solo apuntaba la nota de cada versión y no siempre lo que había cambiado, así que en la tabla pongo lo principal y los cambios están explicados en el apartado 2.1.2.

Versión / cambio	Puntuación	Observaciones
Primera versión (condiciones básicas de velocidad)	9.63	La primera vuelta completa, sin tener en cuenta los triángulos
Inserción triángulos con punto medio y código optimizado	27.39	Circuito completo haciendo bien ambos tipos de segmentos
Ajuste de valores de las reglas y ajustar bien primer segmento	31.03	Después de tocar velocidades y otros valores para mejorar puntuación, además de mejorar el primer segmento (con variables y valores explícitos para ello)
Optimización del código	34.05	Tras optimizar el código (compactar y mejorar eficiencia)
Versión final (adaptada a los recorridos propuestos en Moodle)	33.81	Mejores recorridos en los circuitos nuevos de Moodle

2.1.4. Ideas descartadas:

- **Cambio de punto justo al tocarlo:** el robot hacía giros muy bruscos y se pasaba del punto por la inercia. En mi código el robot empieza a girar antes de estar justo encima del punto.

2.1.5. Pruebas con otros circuitos y último cambio:

Cuando ya tenía la versión que mejor puntuaba en el circuito de la práctica, fui probando los circuitos nuevos que hay en Moodle. Con ellos he visto dos fallos principales:

- **El primer tramo:** la primera recta es la que casi siempre hace regular. El robot empieza lejos y sin alinear, y tarda en meterse en la recta.

- **Los triángulos cerrados:** Hay triángulos que obligan a hacer un giro brusco, y ahí el robot va a mucha velocidad, se abre en la curva y sale peor que con otro tipo de triángulos.

Para mejorarlo hice un último cambio: bajar la velocidad cuando el ángulo es grande. Con esto pierdo un par de segundos al principio y en las curvas, pero el robot entra mejor en ellas y llega más alineado a las rectas, así que esos tramos puntúan más. En concreto, cambié estos valores:

- Con un error de ángulo de más de 15°, la velocidad baja de 2.5 a 2 m/s (regla G1).
- En la fase 1 del arranque del primer objetivo, la velocidad baja de 1.5 a 1 m/s (regla A1).
- En el cambio de destino, la velocidad pasa de 0 a 0.2 (regla C1).

En el circuito de la práctica la nota baja algo con este cambio, pero prefiero un código más todoterreno que uno que solo puntúe bien en un trozo.

2.2. Explicación del código final:

Aquí explico el código de expertSystem.py: qué tiene la clase, qué hace cada función y cuáles son las reglas que he usado.

2.2.1. Estructura y variables de la clase:

Las variables de la clase son:

- **objetivoActual:** el objetivo que está recorriendo el robot. Lo rellena setObjetivo.
- **estado_inicio:** vale True mientras el robot va hacia el primer punto del tramo (el inicio del segmento o el punto medio del triángulo). Cuando está lo bastante cerca pasa a False y el destino pasa a ser el punto final. setObjetivo la pone en True cada vez que llega un objetivo nuevo.
- **primer_objetivo_global:** vale True solo al principio del todo. Sirve para controlar el arranque del primer tramo, porque el robot empieza lejos y sin estar alineado. Cuando ya está alineado pasa a False y no se vuelve a usar, ya que en el resto de objetivos el robot empieza justo donde acabó el anterior.

2.2.2. Funciones auxiliares y geométricas:

Estas son las funciones que he añadido, en el orden en que se usan dentro de tomarDecision. De cada una explico qué hace, cómo lo hace y por qué la necesito.

setObjetivo(self, objetivo)

Qué hace: guarda en objetivoActual el objetivo que le da el simulador y pone estado_inicio en True.

```
#Guarda en objetivoActual el objetivo al que tiene que moverse el robot.
#También pone estado_inicio a True, así en cada objetivo nuevo el robot va primero al inicio
def setObjetivo(self, objetivo):
    self.objetivoActual = objetivo
    self.estado_inicio = True
```

Por qué: el simulador llama a esta función cada vez que el robot termina un objetivo, para darle el siguiente. Al poner estado_inicio en True, en cada objetivo nuevo el robot empieza yendo al primer punto (el inicio del segmento o el punto medio del triángulo) y después al punto final. La plantilla solo guardaba el objetivo. No toco primer_objetivo_global a propósito, porque el arranque especial solo hace falta una vez, en el primer objetivo.

inicio(poseRobot)

Qué hace: devuelve la posición X, la posición Y y la orientación del robot, que son los elementos 0, 1 y 2 de poseRobot.

```
#   Saca de la pose del robot su posición (x, y) y su orientación en grados.
def inicio(self, poseRobot):
    xrobot = poseRobot[0]
    yrobot = poseRobot[1]
    orientacion = poseRobot[2]
    return xrobot, yrobot, orientacion
```

Por qué: la pose llega con 5 valores y solo necesito esos tres. Los saco con índices y así tomarDecision queda más limpio.

tipo_recorrido()

Qué hace: devuelve el getType() del objetivo actual: 1 si es un segmento y 2 si es un triángulo.

```
#   Devuelve el tipo del objetivo actual: 1 si es un segmento y 2 si es un triángulo
def tipo_recorrido(self):
    tipo = self.objetivoActual.getType()
    return tipo
```

Por qué: los dos tipos de objetivo no se recorren igual. En el segmento el primer destino es el inicio y en el triángulo es el punto medio. Con esta función el código lo decide solo, sin que yo tenga que decirle qué tramo es cada uno, así que vale para cualquier circuito.

distancia(xiniobj, xrobot, yiniobj, yrobot)

Qué hace: recibe la posición del punto de destino y la del robot, y calcula la diferencia en X (difx), la diferencia en Y (dify), la distancia entre los dos y el ángulo hacia el punto (llamando a funcionangulo (lo explico más adelante)) y devuelvo esos cuatro valores.

```
def distancia(self, xiniobj, xrobot, yiniobj, yrobot):
    difx = xiniobj - xrobot
    dify = yiniobj - yrobot
    distancia_objetivo = abs(difx)+abs(dify)
    angulo_objetivo = self.funcionangulo(difx, dify) # Sacamos el ángulo con la función de arriba
    return difx, dify, distancia_objetivo, angulo_objetivo
```

Por qué: así junto en un solo sitio los cálculos que hacen falta en cada instante y no repito código (antes tenía una función para el inicio y otra para el final que hacían casi lo mismo).

funcionangulo(difx, dify)

Qué hace: calcula el ángulo en el que está el punto de destino visto desde el robot.

```python
def funcionangulo(self, difx, dify):
    # Uso trigonometría basica: tan(angulo) = dify / difx, o sea: angulo = arctan(dify / difx).
    # Tengo que usar atan2 porque tiene en cuenta los signos de dify y difx y funciona hacia cualquier lado
    angulo = math.atan2(dify, difx) # Sale en radianes

    # Lo pasamos a grados porque la orientación del robot también está en grados
    angulo_objetivo = angulo * 180/math.pi
    return angulo_objetivo
```

Cómo: con trigonometría básica, la tangente del ángulo es dify / difx, así que el ángulo es el arcotangente. Uso math.atan2, que da el ángulo en radianes, y lo multiplico por 180/π para pasarlo a grados.

Por qué: uso atan2 y no atan porque tiene en cuenta los signos de dify y difx, y así da bien el ángulo hacia cualquier lado (entre -180° y 180°). Lo paso a grados porque la orientación del robot también viene en grados y así se pueden comparar directamente.

calculo_diferencia(angulo_objetivo, orientacion)

Qué hace: calcula cuánto tiene que girar el robot para mirar al punto: el ángulo del objetivo menos la orientación, dejado siempre entre -180° y 180°.

```python
# Calcula cuánto tiene que girar el robot para mirar al punto (ángulo del objetivo menos orientación)
#  Si es positivo el punto está a la izquierda del robot y si es negativo, a la derecha
def calculo_diferencia(self, angulo_objetivo, orientacion):
    diferencia  = angulo_objetivo - orientacion
    while diferencia > 180:
        diferencia -= 360 # Si se pasa de 180 le restamos 360 (por ejemplo, 340 pasa a ser -20)
    while diferencia < -180:
        diferencia += 360 # Si es menor de -180 le sumamos 360
    return diferencia
```

Cómo: resta los dos ángulos y, mientras el resultado sea mayor de 180, le resta 360; mientras sea menor de -180, le suma 360.

Por qué: Al principio, cuando el robot daba muchas vueltas en un sentido, el angulo no paraba de crecer (360, 720...). Sin este ajuste en el que lo mantengo entre +180° y -180° la resta puede dar números enormes y el robot giraría por el lado largo. El signo me dice hacia qué lado girar y el valor cuánto, y es el dato que usan las reglas.

2.2.3. Función iraobjetivo():

Antes de empezar: las reglas del sistema tienen un código con una letra según la función donde están: T en tomarDecision, y G (giro), A (arranque) y C (cambio de destino) en iraobjetivo, seguida de un número (T1, G1, A1...). Más abajo hay una lista con lo que hace cada una y por qué la puse, y además cada regla está marcada en los comentarios del código co nsu código (T1, G1...), así se ve fácilmente de cuál hablo en cada momento.

iraobjetivo(distancia_objetivo, diferencia) es la función más grande y la que decide la velocidad del robot (el motor del sistema). Recibe la distancia al punto de destino y el error de ángulo (dif) y devuelve dos valores: x, que es la velocidad lineal (V), e y, que es la velocidad angular (W). Está dividida en tres partes, en este orden:

- **Giro y velocidad según el error de ángulo** (reglas G1 a G5).
- **Arranque especial del primer objetivo** (reglas A1 y A2), que puede bajar la velocidad.

- **Cambio de destino** cuando el robot está cerca del punto (regla C1).

El orden importa: primero saco V y W con el ángulo, después el arranque puede bajar la velocidad, y al final compruebo si hay que cambiar de destino. En las reglas, “dif” es el error de ángulo (ángulo del objetivo menos orientación, entre -180 y 180) y “d” es la distancia |ΔX| + |ΔY| al punto de destino.

- **G1. Giro grande:** Si |dif| > 15°: V = 2 y W = 0.5 (con el signo de dif). **Por qué:** Si está muy desalineado necesita girar fuerte. Le bajo la velocidad para que la curva sea cerrada (con V = 2 y W = 0.5).
- **G2. Giro medio:** Si 10° < |dif| ≤ 15°: V = 2.5 y W = 0.3. **Por qué:** Es un paso intermedio entre G1 y G3 para no pasar de golpe de ir lento a ir rápido.
- **G3. Giro pequeño:** Si 0.3° < |dif| ≤ 10°: V = 3 y W = 0.1. **Por qué:** Ya está casi alineado, así que corrige despacio y no pierde velocidad. Un W pequeño evita pasarse, porque el robot tarda en dejar de girar
- **G4. Alineado:** Si |dif| ≤ 0.3°: V = 3 y W = 0. **Por qué:** Va recto a la velocidad máxima. Con ese margen no se mueve de un lado a otro por errores muy pequeños.
- **G5. Sentido del giro:** Si dif es positivo, W es positivo; si es negativo, W es negativo. **Por qué:** Como dif está entre -180 y 180, el signo le dice hacia qué lado está el objetivo y el robot siempre gira por el lado corto.
- **A1. Arranque, fase 1:** Solo en el primer objetivo: si d ≤ 3.5, cambia el destino al punto final; si 3.5 < d ≤ 5 va a V = 1. **Por qué:** Al principio el robot viene de lejos y sin alinear. Entrar rápido al punto de inicio hace que se pase, así que se frena un poco antes.
- **A2. Arranque, fase 2:** Solo en el primer objetivo, ya con el punto final como destino: si |dif| > 10° mantiene V = 1; si no, pone primer_objetivo_global en False y ya va a toda velocidad. **Por qué:** Es la regla que quita la desviación del primer tramo: hasta que no está alineado con la recta no se deja que acelere a 3 m/s.
- **C1. Cambio de destino:** Cuando d ≤ 2 y todavía está yendo al inicio / punto medio, pasa a ir al punto final. En ese instante pone V = 0.2 una sola vez. **Por qué:** Cambiar antes de llegar hace la curva más suave. El V = 0.2 de ese instante casi no se nota, porque el robot cambia de velocidad despacio (1 m/s²).

Los valores de las reglas (15°, 10° y 0.3°, y las velocidades de cada una) los he ido ajustando con pruebas. La idea general es que cuanto mayor es el error de ángulo, más giro y menos velocidad, porque el radio de la curva es mayor a mayor velocidad, y para giros rápidos, velocidades rápidas no funcionan bien.

2.2.4. Función tomarDecision():

tomarDecision(poseRobot) es la función que llama el simulador en cada instante y la que tiene que devolver la velocidad lineal y la angular.

En esta función tenemos dos reglas:

- **T1. Sin objetivo:** Si objetivoActual es None, devuelve V = 0 y W = 0. **Por qué:** Para que el robot no haga nada si todavía no tiene objetivo (una especie de caso base).
- **T2. Destino:** Si el tramo es un segmento, va primero al inicio y luego al fin. Si es un triángulo, va primero al punto medio y luego al fin. **Por qué:** En el triángulo, pasar por el punto medio hace que el robot lo rodee y no entre en el área. En el segmento, ir al inicio asegura que empiece sobre la recta.

Simplemente agrupa todas las funciones anteriores, en este orden:

1. Si no hay objetivo (objetivoActual es None), devuelve (0, 0). Es la regla T1.
2. Llama a inicio para sacar la posición X, la posición Y y la orientación de la pose.
3. Llama a tipo_recorrido para saber si el tramo es un segmento o un triángulo.
4. Elige el punto de destino (regla T2): el inicio (o el punto medio si es triángulo) mientras estado_inicio sea True, y el punto final cuando pase a False.
5. Llama a distancia para sacar difx, dify, la distancia y el ángulo hacia el punto.
6. Llama a calculo_diferencia para sacar el error de ángulo entre -180 y 180.
7. Llama a iraobjetivo con la distancia y el error de ángulo, que aplica las reglas G1 a G5, A1 y A2 y devuelve (V, W). Dentro de iraobjetivo es donde estado_inicio y primer_objetivo_global pueden cambiar.
8. Devuelve (V, W) al simulador, que se los da al robot.

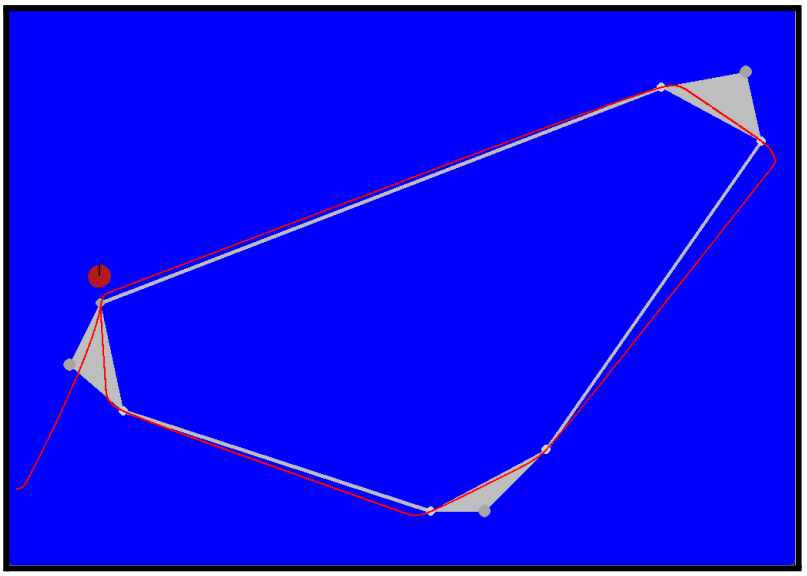
Como esto se repite en cada instante, el robot corrige su rumbo continuamente. No planifica el camino entero: solo mira dónde está y hacia dónde tiene que girar ahora.

2.3. Resultados del modelo final:

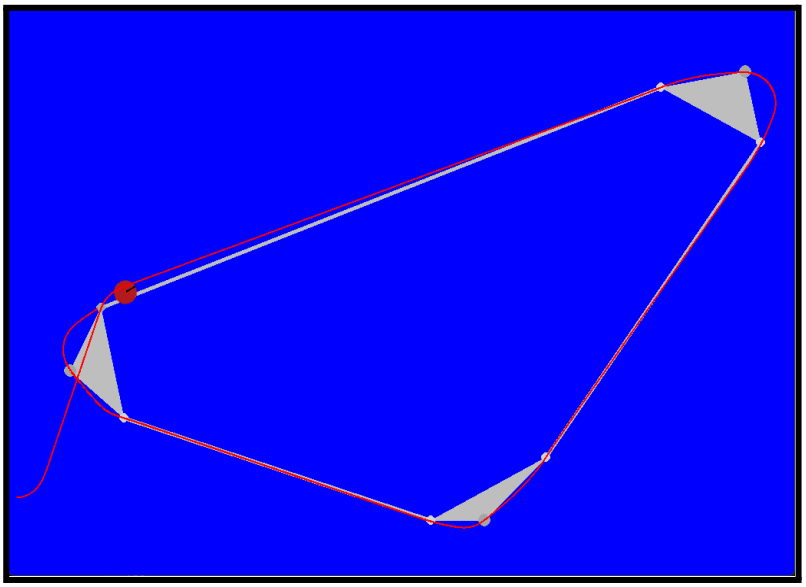
Esta es la tabla con los datos de cada objetivo en la ejecución final (del recorrido1). La nota total de la versión final es de 33.813 puntos.

Objetivo	Tipo	Puntuación de distancia	Tiempo (s)	Nota
1	Segmento	35842.64	37.47	0.629
2	Triángulo	491	9.05	4.859
3	Segmento	28673.09	16.27	5.558
4	Triángulo	495	6.84	8.036
5	Segmento	29888.64	14.01	8.827
6	Triángulo	497	8.17	5.902
Total	-	-	-	33.813

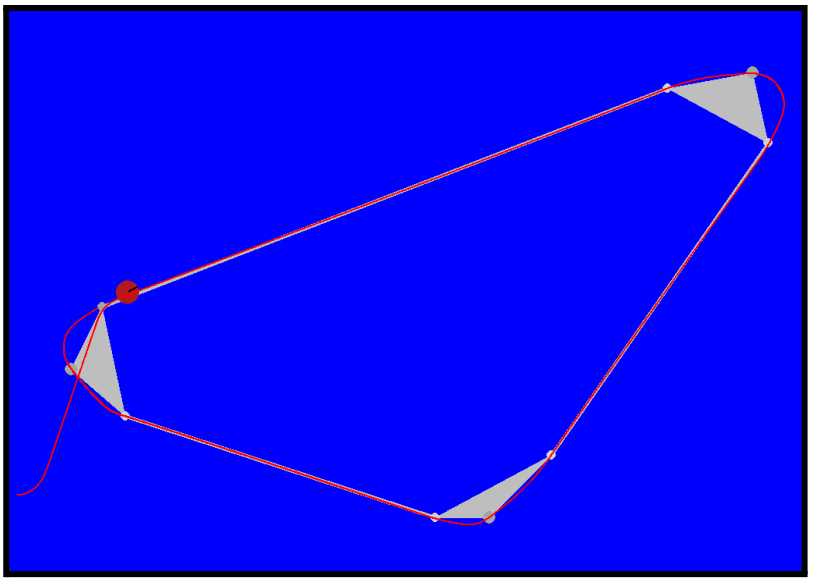
Primera versión del Sistema Experto:



Sistema Experto sin ajustar el primer tramo:



Sistema Experto Final:



2.4. Conclusiones y posibles mejoras:

2.4.1. Ventajas y puntos fuertes:

- **Es rápido:** el robot va a la velocidad máxima de 3 m/s casi todo el recorrido, y como el tiempo se divide al cubo en la nota, eso es lo que más puntos da.
- **Funciona con cualquier posición:** todo se calcula con diferencias entre la posición del robot y la de los puntos, y el tipo de tramo se detecta con getType(), así que vale para cualquier circuito.
- **Es sencillo de entender:** son pocas reglas y cada una hace una cosa concreta.
- **Es todoterreno:** prefiero un código que funcione bien en cualquier circuito a uno ajustado para sacar la máxima nota en un solo trozo.
- **Sigue bien la recta:** una vez alineado, el robot va pegado al camino (ver capturas).

2.4.2. Limitaciones y futuras mejoras:

- **Los valores están ajustados a mano** para este circuito y esta dinámica del robot. En otro circuito podrían no ser los mejores.
- **En los triángulos apunta directo al punto medio**. Se podría apuntar a un punto un poco más separado del vértice para evitar rozar el interior del triangulo.
- **El primer tramo sale regular.** El robot empieza lejos y sin alinear, y tarda en meterse bien en la primera recta aunque tenga el arranque en dos fases.
- **Los triángulos cerrados salen peor.** Si el triángulo obliga a un giro brusco, el robot solo baja la velocidad cuando ya tiene el ángulo grande, no antes de entrar en la curva, y por la inercia se abre. Se podría bajar la velocidad al pasar por cada punto, pero cambiaba toda la práctica.
- **No uso la distancia a la recta** para corregir, solo el ángulo hacia el punto. Medirla permitiría seguir la recta con más precisión.
- **El arranque es un caso especial** (primer_objetivo_global). Estaría mejor una regla general que sirviera también para cuando el robot empieza mal colocado en cualquier objetivo.

3. Ejercicio 2: Sistema experto difuso

En el ejercicio 2 se hace lo mismo que en el 1, pero con lógica difusa en vez de reglas if-then, usando la librería fuzzy-expert. La idea que he seguido es partir de mi sistema experto, que ya funciona bien, y pasar sus reglas a reglas difusas en vez de empezar de cero.

La diferencia principal es que en el sistema experto las reglas cambian de golpe (con un error de 15° o menos la velocidad es una y con más es otra), y en el difuso el cambio es gradual: cuando el error está entre dos rangos, se mezclan las dos reglas. Lo que no cambia es la parte de los estados (estado_inicio, primer_objetivo_global y los cambios de punto), que está igual que en el sistema experto.

Con la versión final saco 32.585 puntos en el recorrido 1. Al igual que en el ejercicio 1, primero cuento cómo fue el trabajo, después explico el código, y por último pongo los resultados y las conclusiones.

3.1. Evolución del trabajo:

Aquí explico cómo ha ido mejorando el sistema difuso, desde la primera versión hasta la final.

3.1.1. Primera versión y avances iniciales:

Para hacer mi primera versión difusa me basé en la plantilla del profesor y en el ejemplo de la propina (calculaPropina.py), para entender cómo funciona la librería. Tenía 4 variables difusas (el error de ángulo con 5 conjuntos, la distancia al punto con 3 conjuntos, y las velocidades V y W) y 6 reglas: 5 para girar y avanzar según el error, y una que bajaba la velocidad al estar cerca del punto. El cambio de destino lo hacía a menos de 2 metros. Con esta versión saqué 11.35 puntos y el robot tambaleaba mucho. Al revisarla vi varias cosas:

- **El conjunto “Alineado” era muy ancho** (de -10° a 10°). Con un error de pocos grados el robot casi no corregía el rumbo y se desviaba de la recta, mientras que en el sistema experto corrige desde 0.3°.
- **No tenía las velocidades ni el arranque del sistema experto:** los valores de V y W eran otros y no había arranque especial en el primer objetivo.
- **En los triángulos** entraba dentro del área durante mucho tiempo y perdía puntos.

3.1.2. Proceso de optimización:

Para mejorar la primera versión no empecé de cero: Aproveché que ya habia optimizado mi sistema Experty y cogí la lógica de este, que ya funcionaba bien, y la pasé al sistema difuso. Este paso simplemente consistió en traducir las reglas y los valores de velocidad y giro a variables, conjuntos y reglas difusas. Estos fueron los cambios:

- **Pasar los estados del sistema experto:** el cambio de destino con la distancia |ΔX| + |ΔY| (a 3.5 en el primer objetivo y a 2 en el resto) y el arranque en dos fases del primer objetivo. En el difuso para el arranque utilizo una variable de entrada que vale 1 únicamente en esa fase.
- **Usar las mismas velocidades y giros:** los conjuntos de salida de V y W son los mismos valores del sistema experto

- **Conjuntos del error con los mismos rangos:** alineado (±0.5°), pequeño (hasta 10°), medio (de 12° a 14°) y grande (a partir de 16°). El conjunto Alineado pasó de ±10° a ±0.5°.

Con estos cambios la nota en el recorrido 1 subió de 11.35 a 32.585

3.1.3. Historial de puntuaciones:

Del ejercicio 2 tengo apuntadas dos puntuaciones: la de la primera versión y la de la versión final.

Versión	Puntuación	Observaciones
Primera versión difusa (6 reglas)	11.35	Conjunto Alineado muy ancho y frenaba cerca de los puntos; el robot tambaleaba
Versión final (15 reglas, lógica del sistema experto)	32.585	Estados, variables y velocidades del sistema experto traducidos a reglas difusas

3.1.4. Ideas descartadas:

Como me he basado mayoritariamente en el sistema experto , no han habido ideas descartadas, ya que me he limitado a traducir las ideas del experto al fuzzy. Como ya tenia optimizado el sistema experto, no ha sido necesario optimizar el difuso

3.2. Explicación del código final:

Aquí explico el código de fuzzyExpert.py: qué tiene la clase, qué hace cada función, y cuáles son las variables, los conjuntos y las reglas difusas.

3.2.1. Estructura y variables de la clase:

Las variables de la clase son:

- **objetivoActual:** el objetivo que está recorriendo el robot. Lo rellena setObjetivo.
- **estado_inicio y primer_objetivo_global:** son las mismas que en el sistema experto. estado_inicio vale True mientras el robot va hacia el primer punto del tramo, y primer_objetivo_global vale True solo al principio de todo, para el arranque del primer objetivo.
- **variables, reglas y modelo:** las variables difusas, las 15 reglas y el motor de inferencia

3.2.2. Funciones auxiliares:

En el sistema experto tenía varias funciones auxiliares (inicio, tipo_recorrido, distancia, funcionangulo y calculo_diferencia). En el difuso solo tengo setObjetivo, y esos mismos cálculos los hago directamente dentro de tomarDecision, ya que al haber tantas variables del sistema difuso, he pensado que la mejor opción es reducir la cantidad de funciones para favorecer la lectura del código.

setObjetivo(self, obj)

Qué hace: guarda el objetivo en objetivoActual y pone estado_inicio en True, igual que en el sistema experto.

```
# función setObjetivo
#   Guarda el objetivo y pone estado_inicio a True, igual que en el sistema experto
def setObjetivo(self, obj):
    self.objetivoActual = obj
    self.estado_inicio = True
```

Por qué: el simulador llama a esta función cada vez que el robot termina un objetivo. Con estado_inicio en True, en cada objetivo nuevo el robot va primero al primer punto del tramo y después al punto final.

3.2.3. Variables y conjuntos difusos:

El sistema tiene dos variables de entrada y dos de salida:

Variable	Tipo	Rango	Conjuntos
error_angulo	Entrada	-180° a 180°	Izq_Grande, Izq_Media, Izq_Pequena, Alineado, Der_Pequena, Der_Media, Der_Grande
arranque	Entrada	0 a 1	No, Si
v_lineal	Salida	0 a 3.5 m/s	Arranque (1), Lenta (2), Media (2.5), Rapida (3)
v_angular	Salida	-0.8 a 0.8 rad/s	Der_Fuerte (-0.5), Der_Media (-0.3), Der_Suave (-0.1), Recto (0), Izq_Suave (0.1), Izq_Media (0.3), Izq_Fuerte (0.5)

```
self.variables = {
    "error_angulo": FuzzyVariable(
        universe_range=(-180, 180),
        terms={
            "Izq_Grande":  [(14, 0), (16, 1), (180, 1)],
            "Izq_Media":   [(10, 0), (12, 1), (14, 1), (16, 0)],
            "Izq_Pequena": [(0, 0), (0.5, 1), (10, 1), (12, 0)],
            "Alineado":    [(-0.5, 0), (0, 1), (0.5, 0)],
            "Der_Pequena": [(-12, 0), (-10, 1), (-0.5, 1), (0, 0)],
            "Der_Media":   [(-16, 0), (-14, 1), (-12, 1), (-10, 0)],
            "Der_Grande":  [(-180, 1), (-16, 1), (-14, 0)],
        }
    ),
    # Vale 1 solo durante el arranque del primer objetivo (como primer_objetivo_global en el sistema experto)
    "arranque": FuzzyVariable(
        universe_range=(0, 1),
        terms={
            "No": [(0, 1), (1, 0)],
            "Si": [(0, 0), (1, 1)],
        }
    ),
    # Los valores de V y W son los mismos que usa el sistema experto
    "v_lineal": FuzzyVariable(
        universe_range=(0, 3.5),
        terms={
            "Arranque": [(0.5, 0), (1.0, 1), (1.5, 0)],
            "Lenta":    [(1.5, 0), (2.0, 1), (2.5, 0)],
            "Media":    [(2.0, 0), (2.5, 1), (3.0, 0)],
            "Rapida":   [(2.5, 0), (3.0, 1), (3.5, 0)],
        }
    ),
    "v_angular": FuzzyVariable(
        universe_range=(-0.8, 0.8),
        terms={
            "Der_Fuerte": [(-0.6, 0), (-0.5, 1), (-0.4, 0)],
            "Der_Media":  [(-0.4, 0), (-0.3, 1), (-0.2, 0)],
            "Der_Suave":  [(-0.2, 0), (-0.1, 1), (0.0, 0)],
            "Recto":      [(-0.1, 0), (0.0, 1), (0.1, 0)],
            "Izq_Suave":  [(0.0, 0), (0.1, 1), (0.2, 0)],
            "Izq_Media":  [(0.2, 0), (0.3, 1), (0.4, 0)],
            "Izq_Fuerte": [(0.4, 0), (0.5, 1), (0.6, 0)],
        }
    ),
}
```

El error angular es el mismo que en el sistema experto (ángulo del objetivo menos orientación, entre -180° y 180°): es positivo si el punto está a la izquierda del robot y negativo si está a la derecha. La variable arranque vale 1 solo en el arranque del primer objetivo, que es cuando el sistema experto bajaba la velocidad a 1 m/s.

Los rangos de v_lineal (hasta 3.5) y v_angular (hasta ±0.8) son un poco más grandes que los límites del robot a propósito. Así en los conjuntos de los extremos (3 m/s y 0.5 rad/s) el resultado sale justo en el valor que quiero. Si se cortaran en el borde, saldrían a velocidades más bajas. Ademas, el robot ya limita la velocidad a 3 m/s.

Conjuntos difusos del error angular

Conjunto	Puntos (ángulo, pertenencia)	Qué significa
Alineado	(-0.5, 0), (0, 1), (0.5, 0)	Está mirando al punto
Izq_Pequena	(0, 0), (0.5, 1), (10, 1), (12, 0)	Poco desalineado
Izq_Media	(10, 0), (12, 1), (14, 1), (16, 0)	Bastante desalineado
Izq_Grande	(14, 0), (16, 1), (180, 1)	Muy desalineado

3.2.4. Reglas difusas:

Son 15 reglas: 7 de giro, 7 de velocidad y 1 de arranque. Son las reglas G1 a G5, A1 y A2 del sistema experto, pero cada una se aplica en mayor o menor grado según lo cerca que esté el error de su conjunto.

- **Regla 1:** Si el error es Izq_Grande, entonces v_angular es Izq_Fuerte (0.5). Equivale a G1 (giro grande).
- **Regla 2:** Si el error es Izq_Media, entonces v_angular es Izq_Media (0.3). Equivale a G2 (giro medio).
- **Regla 3:** Si el error es Izq_Pequena, entonces v_angular es Izq_Suave (0.1). Equivale a G3 (giro pequeño).
- **Regla 4:** Si el error es Alineado, entonces v_angular es Recto (0). Equivale a G4 (alineado).
- **Regla 5 a 7:** Si el error es Der_Pequena / Der_Media / Der_Grande, entonces v_angular es Der_Suave / Der_Media / Der_Fuerte (-0.1 / -0.3 / -0.5). Equivale a G5 (sentido del giro).
- **Regla 8:** Si el error es Izq_Grande Y arranque es No, entonces v_lineal es Lenta (2). Equivale a G1.
- **Regla 9:** Si el error es Izq_Media Y arranque es No, entonces v_lineal es Media (2.5). Equivale a G2.
- **Regla 10:** Si el error es Izq_Pequena Y arranque es No, entonces v_lineal es Rapida (3). Equivale a G3.
- **Regla 11:** Si el error es Alineado Y arranque es No, entonces v_lineal es Rapida (3). Equivale a G4.
- **Regla 12 a 14:** Si el error es Der_Pequena / Der_Media / Der_Grande Y arranque es No, entonces v_lineal es Rapida / Media / Lenta. Equivale a G3, G2 y G1.
- **Regla 15:** Arranque es Si, entonces v_lineal es Arranque (1). Equivale a A1 y A2.

Las reglas de velocidad llevan “Y arranque es No” para que no se mezclen con la del arranque: cuando arranque vale 1 solo manda la regla 15 (1 m/s), y cuando vale 0 mandan las de velocidad.

Siempre se activa al menos una regla de giro y una de velocidad, porque los conjuntos del error se solapan y cubren todos los valores de -180° a 180°. Esto es necesario para que la librería siempre tenga un resultado que calcular.

```
# 2. REGLAS DIFUSAS
# Son las mismas reglas que iraobjetivo (G1 a G5), pero cada una se aplica "un poco" según lo cerca que esté
giros = [
    ("Izq_Grande", "Izq_Fuerte"),
    ("Izq_Media", "Izq_Media"),
    ("Izq_Pequena", "Izq_Suave"),
    ("Alineado", "Recto"),
    ("Der_Pequena", "Der_Suave"),
    ("Der_Media", "Der_Media"),
    ("Der_Grande", "Der_Fuerte"),
]
velocidades = [
    ("Izq_Grande", "Lenta"),
    ("Izq_Media", "Media"),
    ("Izq_Pequena", "Rapida"),
    ("Alineado", "Rapida"),
    ("Der_Pequena", "Rapida"),
    ("Der_Media", "Media"),
    ("Der_Grande", "Lenta"),
]
# Reglas de dirección: cuanto más error, más fuerte se gira
self.reglas = [
    FuzzyRule(
        premise=[("error_angulo", error)],
        consequence=[("v_angular", giro)]
    )
    for error, giro in giros
]
# Reglas de velocidad (fuera del arranque): cuanto más error, menos velocidad
self.reglas += [
    FuzzyRule(
        premise=[("error_angulo", error), ("AND", "arranque", "No")],
        consequence=[("v_lineal", vel)]
    )
    for error, vel in velocidades
]
# Regla del arranque (A1 y A2): en el arranque del primer objetivo se va a 1 m/s
self.reglas.append(
    FuzzyRule(
        premise=[("arranque", "Si")],
        consequence=[("v_lineal", "Arranque")]
    )
)
```

3.2.5. Motor de inferencia:

Uso el método de Mamdani con el centro de gravedad. En concreto:

- **Implicación:** “Rc”, que en esta librería es el mínimo (la de Mamdani).
- **AND:** mínimo. **OR:** máximo (aunque solo uso AND).
- **Composición:** max-min. **Unión de las reglas:** máximo.
- **Defuzzificación:** centro de gravedad (“cog”), que da un valor intermedio cuando se activan dos reglas a la vez.

```
# 3. MOTOR DE INFERENCIA
self.modelo = DecompositionalInference(
    and_operator="min",
    or_operator="max",
    implication_operator="Rc",
    composition_operator="max-min",
    production_link="max",
    defuzzification_operator="cog",
)
```

Es la misma configuración que usaba en mi primera versión, que no daba fallos con la librería, así que no la he cambiado.

3.2.6. Función tomarDecision():

Hace lo mismo que la de mi sistema experto, pero al final llama al sistema difuso en vez de a iraobjetivo:

- Si no hay objetivo, devuelve (0, 0).
- Elige el punto de destino igual que el sistema experto (regla T2) y calcula la distancia (|difx| + |dify|) y el error de ángulo entre -180° y 180°.
- Controla los estados con la misma lógica que iraobjetivo: en el primer objetivo, a 3.5 o menos pasa a ir al punto final, y mientras se acerca (a 5 o menos) o hasta que el error baja de 10°, pone arranque = 1. En cualquier objetivo, a 2 o menos del primer punto cambia el destino.
- Llama al sistema difuso (self.modelo) con el error de ángulo y el valor de arranque, y se queda con V (v_lineal) y W (v_angular).

```python
def tomarDecision(self, poseRobot):
    V = W = 0

    # Si todavía no hay objetivo, el robot se queda parado
    if self.objetivoActual is None:
        return (V, W)

    xrobot, yrobot, orientacion = poseRobot[0], poseRobot[1], poseRobot[2]

    # Punto al que vamos: igual que en el sistema experto
    if self.objetivoActual.getType() == 1:
        punto = self.objetivoActual.getInicio() if self.estado_inicio else self.objetivoActual.getFin()
    else:
        punto = self.objetivoActual.getMedio() if self.estado_inicio else self.objetivoActual.getFin()

    # Distancia (suma de |difx| y |dify|, como en el sistema experto) y error de ángulo
    difx = punto[0] - xrobot
    dify = punto[1] - yrobot
    distancia = abs(difx) + abs(dify)
    error_ang = math.degrees(math.atan2(dify, difx)) - orientacion
    while error_ang > 180:
        error_ang -= 360
    while error_ang < -180:
        error_ang += 360

    # Control de estados: es la misma lógica que tiene iraobjetivo, solo que aquí no cambia
    # la velocidad directamente, sino la variable "arranque" que usa el sistema difuso
    arranque = 0
    if self.primer_objetivo_global:
        if distancia <= 3.5:
            self.estado_inicio = False
        if self.estado_inicio:
            if distancia <= 5:
                arranque = 1
        else:
            if abs(error_ang) > 10:
                arranque = 1
            else:
                self.primer_objetivo_global = False

    # Cambio de destino cuando estamos a 2 o menos del primer punto del tramo
    if distancia <= 2 and self.estado_inicio:
        self.estado_inicio = False

    # Uso el modelo difuso para obtener V y W
    resultado, _ = self.modelo(
        variables=self.variables,
        rules=self.reglas,
        error_angulo=error_ang,
        arranque=arranque,
    )
    V = resultado.get("v_lineal", 0.0)
    W = resultado.get("v_angular", 0.0)
    return (V, W)
```

3.2.7. Cosas que tener en cuenta con la librería:

- **La librería devuelve dos cosas** (el resultado y un dato extra), por eso uso resultado, _ = self.modelo(...), como en el ejemplo de la propina. Los valores de V y W se sacan del resultado con v_lineal y v_angular.
- **Que no falle:** he mantenido la forma de usar la librería de mi primera versión (FuzzyVariable, FuzzyRule y DecompositionalInference) para que no diese fallos.

3.3. Resultados del modelo final:

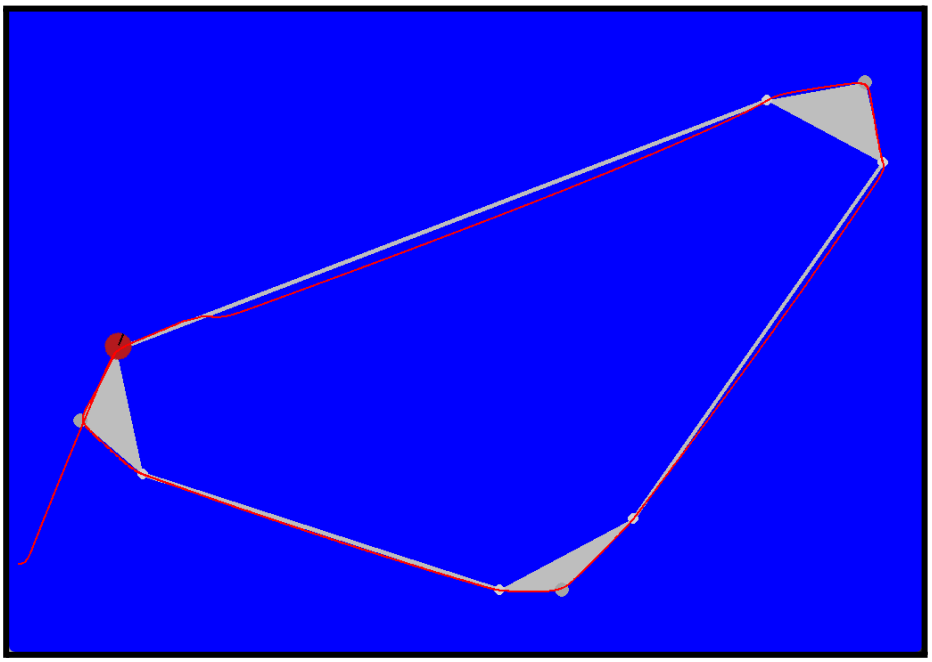
Para el Recorrido 1:

Objetivo	Tipo	Puntuación de distancia	Tiempo (s)	Nota
1	Segmento	16558.640	37.578	0.288
2	Triángulo	494	9.022	4.918
3	Segmento	10268.827	16.389	1.953
4	Triángulo	497	6.872	8.020
5	Segmento	37864.940	14.120	10.954
6	Triángulo	500	7.803	6.452
Total				32.585

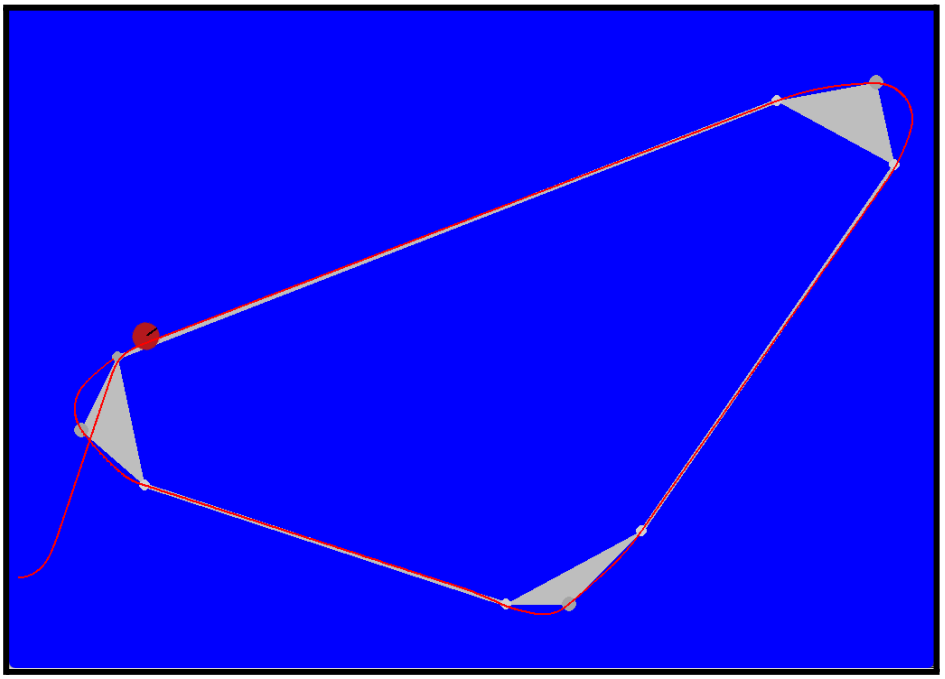
Comparación de puntuaciones totales en el recorrido 1:

Versión	Puntuación
Mi primera versión difusa	11.35
Sistema difuso final	32.59
Sistema experto (ejercicio 1)	33.81

Primera versión difusa:



Última versión difusa:



3.4. Conclusiones y posibles mejoras:

3.4.1. Ventajas y puntos fuertes:

- **Reutiliza la lógica del sistema experto:** mismos estados, mismos umbrales y mismos valores de velocidad y giro, así que se entiende a partir del ejercicio 1.
- **Cambios suaves:** al mezclar reglas cuando el error está entre dos rangos, el robot no cambia de velocidad ni de giro de golpe.
- **Fácil de hacer a partir del ejercicio 1:** una vez hecho el sistema experto, solo tenia que traducir las reglas y los valores a variables y reglas difusas.

3.4.2. Limitaciones y mejoras futuras:

- **Es sensible a los valores.** El tamaño de los conjuntos influye mucho, y cambios pequeños hacen variar bastante la puntuación.
- **Mantiene las limitaciones del sistema experto:** el primer tramo sale regular, los triángulos cerrados salen peor y apunta directo al punto medio.
- **Los resultados dependen del circuito y de los valores:** hay que probarlo en varios circuitos y ajustar los conjuntos.
- **Es más complejo:** hay más cosas que ajustar (conjuntos y reglas) y es más difícil de entender de un vistazo que las reglas if-then (aun evitando tantas funciones e intentando facilitar la lectura).

- **Puede ir más lento:** la librería va añadiendo cada valor de entrada nuevo a su lista de valores (el universo), así que confomre avanza la simulación cada cálculo tarda un poco más y el robot puede tambalear.

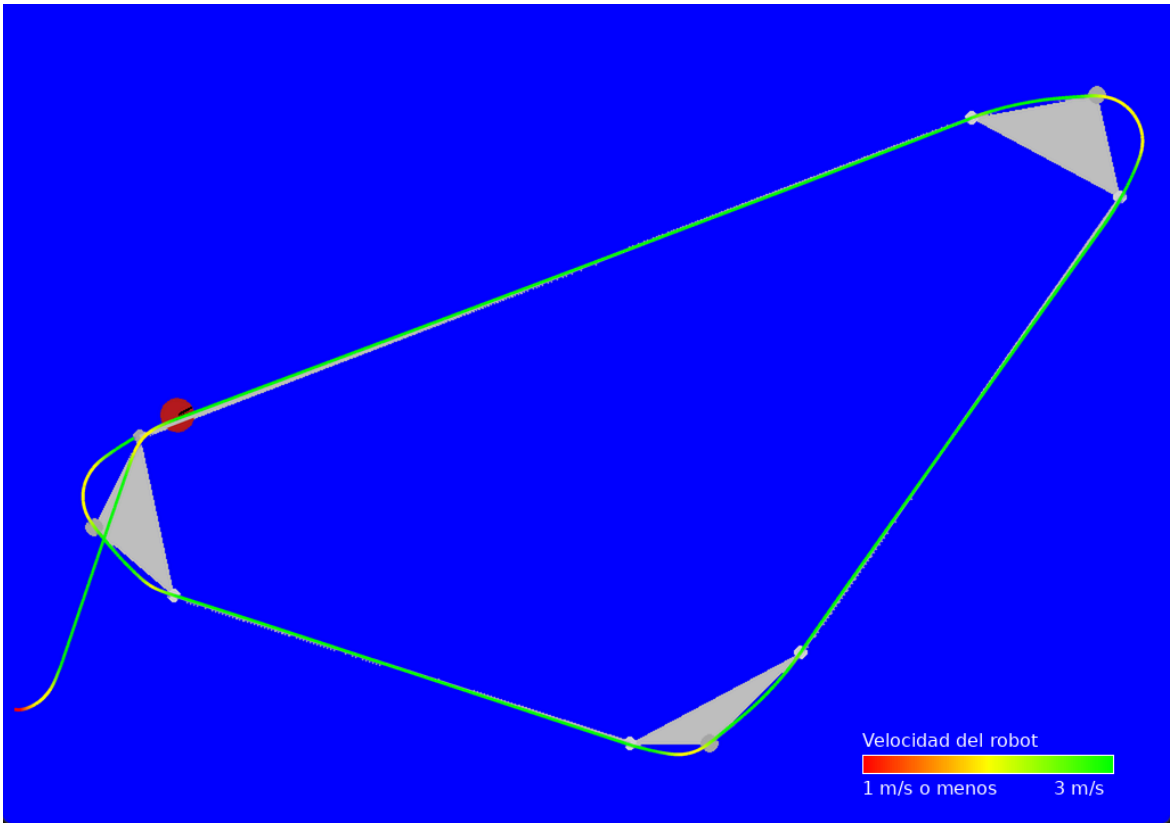
4. Comparación entre los dos ejercicios y conclusiones generales

Los dos ejercicios resuelven el mismo problema con la misma lógica de fondo; lo que cambia es la forma de aplicarla.

Los dos comparten las mismas limitaciones, porque vienen de la lógica común: el primer tramo sale regular, los triángulos cerrados salen peor y la distancia que uso no es la real.

Haciendo los dos ejercicios he visto que el sistema experto es más fácil de entender y de ajustar, y que, una vez hecho, pasarlo al difuso consistió en traducir sus reglas y sus valores. El difuso da cambios más suaves entre reglas, pero tiene más cosas que ajustar y depende de cómo se use la librería. Además, al “heredar” las limitaciones del experto, noto que no consigue sacar más nota que este, por lo que habría que darle otro enfoque al difuso si quisiéramos que obtenga mayor nota que el experto, y no limitarnos a transcribir la lógica.

Sugerencia de mejora del simulador: una idea que se me ha ocurrido mientras hacía las pruebas es que el color de la línea que dibuja el recorrido del robot dependa de la velocidad a la que iba en cada momento. Por ejemplo, verde si va rápido (cerca de 3 m/s), amarillo si va a una velocidad media y rojo si va despacio, con un cambio gradual entre colores. Así se vería de un vistazo en qué partes del circuito frena el robot (en las curvas o al cambiar de punto, por ejemplo), y se podrían comparar mejor el sistema experto y el difuso. Sé que los archivos del simulador no se pueden modificar, por eso lo dejo solo como sugerencia. Adjunto una imagen en la que se puede ver una idea de cómo sería con el color gradual dependiendo de la velocidad (generada por IA):



5. Datos de puntuaciones

5.1 ExpertSystem (ejercicio 1):

Estas son las líneas que imprime el simulador en la ejecución final:

Puntuación del objetivo 1: 0.6291872951722197.

Puntuación de distancia: 35842.64075911848 en 37.47748565673828 segundos

Puntuación del objetivo 2: 4.859028632213102.

Puntuación de distancia: 491 en 9.052313327789307 segundos

Puntuación del objetivo 3: 5.558215889969629.

Puntuación de distancia: 28673.097892929636 en 16.27877974510193 segundos

Puntuación del objetivo 4: 8.03608333720712.

Puntuación de distancia: 495 en 6.848386526107788 segundos

Puntuación del objetivo 5: 8.827931803357624.

Puntuación de distancia: 29888.649910724227 en 14.015821933746338 segundos

Puntuación del objetivo 6: 5.9027402866204435.

Puntuación de distancia: 497 en 8.175956726074219 segundos

Puntuación total: **33.81318724454014**

5.2 FuzzySystem (ejercicio 2):

Puntuación del objetivo 1: 0.288409430547647.

Puntuación de distancia: 16558.63999961615 en 37.57788395881653 segundos

Puntuación del objetivo 2: 4.918334977942246.

Puntuación de distancia: 494 en 9.022000551223755 segundos

Puntuación del objetivo 3: 1.9529987691007473.

Puntuación de distancia: 10268.827044160716 en 16.388933897018433 segundos

Puntuación del objetivo 4: 8.01967186908278.

Puntuación de distancia: 497 en 6.872268438339233 segundos

Puntuación del objetivo 5: 10.9538095201203.

Puntuación de distancia: 37864.93984715281 en 14.12019395828247 segundos

Puntuación del objetivo 6: 6.451919102249426.

Puntuación de distancia: 500 en 7.803199529647827 segundos

Puntuación total: **32.58514366904315**

6. Uso de la IA

En este proyecto he utilizado la Inteligencia Artificial únicamente como una herramienta de apoyo y aprendizaje. A lo largo del desarrollo de la práctica, me ha servido para resolver dudas concretas, entender mejor algunos conceptos teóricos y encontrar el motivo de los errores que iban apareciendo en el código durante las pruebas.

En ningún momento se ha utilizado Inteligencia Artificial para que realice la práctica por su cuenta ni para generar la lógica del robot, sino exclusivamente como una herramienta de consulta personal para orientarme y guiarme cuando me quedaba atascado en algún problema.

7. Bibliografía

1. Documentación de la librería fuzzy-expert: Fuzzy-Expert: Mamdani Fuzzy Inference Systems in Python.
2. Enunciado de la Práctica 1
3. Código proporcionado por el profesorado: simulador (P1Launcher.py, robot.py y objetivo.py), ejemplo de sistema difuso (calculaPropina.py), plantillas de expertSystem.py y fuzzyExpert.py. y segmentos varios.

8. Repositorio de GitHub

Le he dedicado mucho tiempo a esta práctica y la he ido haciendo semana a semana, probando y cambiando cosas por el camino. Para que se pueda ver cómo ha evolucionado el proyecto, he dejado todo el código en un repositorio de GitHub. Ahí está el historial de cambios (commits), desde las primeras versiones hasta la final:

https://github.com/jorgesancere/Uni-2.1-Razonamiento-y-Representacion

En el repositorio están expertSystem.py y fuzzyExpert.py, el simulador, los circuitos de prueba y un README básico con las instrucciones para ejecutarlo.